

CachePilot

基于 Mooncake 的 KVCache 复用与调度评测系统

真实 Store Benchmark · 可控 Workload 策略评估 · Dashboard 演示

非侵入式 KVCache 评测套件

封装 Mooncake 官方 benchmark, 完成真实 Store verify_write 与 read_perf 实机测试, 并扩展复用与调度评估。

<https://github.com/imperson123/CachePilot>

真实 Store

CSV + 图表

Dashboard

真实 Store 实测

16 组

read_perf 全部成功

峰值带宽

1399.08

MiB/s, 1MB batch=4

正确性验证

0

misses / failures



汇报脉络

项目目标

- ▶ 面向 Mooncake KVCache 赛题，补充可复现实测与策略评估工具。
- ▶ 不侵入 Store、Transfer Engine 或 RDMA 核心路径。
- ▶ 用 CSV、日志、图表和 Dashboard 支撑文档与演示视频。

当前完成内容

模块	完成内容
Store	包装官方 benchmark，完成真实 verify/read_perf 实测
Prefix	可控 workload 下分析 TTFT 改善趋势
Scheduler	对比四类 KV 检索调度策略
Dashboard	提供录屏与截图入口

一句话定位：CachePilot 是 Mooncake 的非侵入式 KVCache benchmark/evaluation suite。

赛题问题定位：KVCache 评测与调度缺口

赛题关注

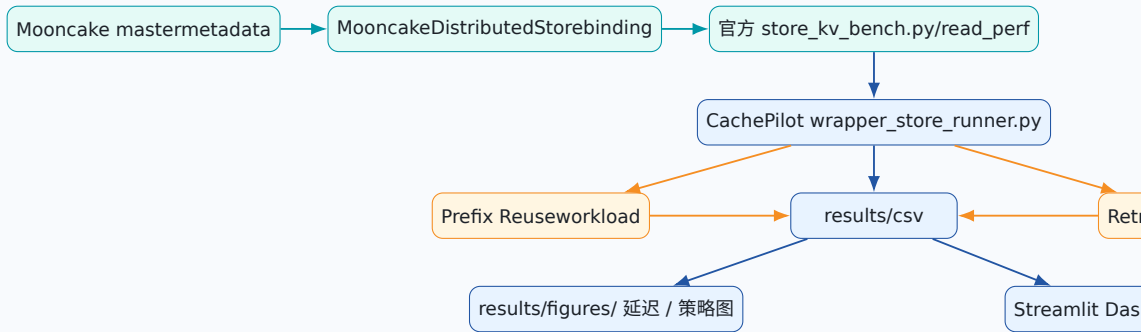
- ▶ KVCache 存储、复用、传输和调度能力。
- ▶ 长上下文 / 多轮对话下的 TTFT、吞吐与 P99。
- ▶ 与 Mooncake Store、HiCache、SGLang / vLLM 生态兼容。

CachePilot 对应

- ▶ 复用官方 Store benchmark，建立真实 I/O 基线。
- ▶ 扩展 Prefix Reuse 与 Retrieval Scheduler 评估。
- ▶ 提供结果表、图表、Dashboard 和运行脚本。

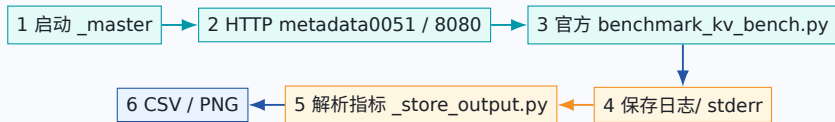


系统总体架构



绿色：真实 Mooncake Store 实机链路；橙色：可控 workload 策略评估；蓝色：结果与展示。

真实 Store Benchmark 数据采集流程



测试场景

verify_write 确认读写正确性; read_perf 覆盖 4KB、64KB、1MB、4MB 与 batch=1/4/8/16。

输出指标

req/s、kv/s、MiB/s、lat_p50、lat_p95、lat_p99、misses 与 verify failures。

实验环境与运行配置

配置项	实验设置
平台	AutoDL 容器实例
GPU	RTX 4090 24GB
CPU / 内存	16 核 / 120GB
系统	Ubuntu 22.04
Python /	3.10 / 11.8
CUDA	
Mooncake 包	non-cuda
协议	TCP

```
export LD_PRELOAD=/usr/lib/x86_64-linux-gnu/libstdc++.so.6
export MOONCAKE_ROOT=/root/autodl-tmp/mooncake_real_run/Mooncake
export HOST_IP=$(hostname -I | awk '{print $1}')

mooncake_master --port=50051 \
  --enable_http_metadata_server=true \
  --http_metadata_server_port=8080

python benchmark/mooncake_store_runner.py \
  --scenario read_perf --io-api plain \
  --value-sizes 4096,65536,1048576,4194304 \
  --batch-sizes 1,4,8,16 --runtime 5
```

功能验证: verify_write 成功

return_code

0

真实 Store 链路可用

MiB/s

38.80

4KB, batch=4

P99

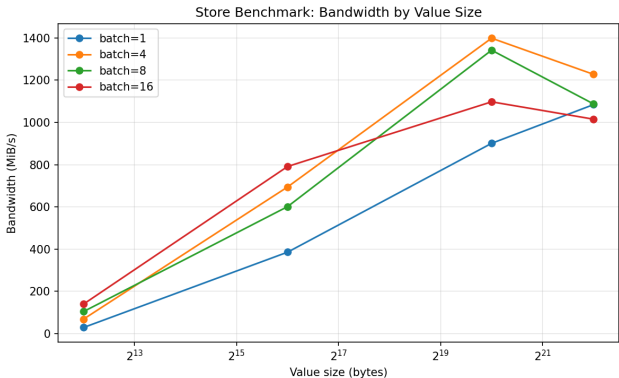
0.781 ms

misses=0, failures=0

value	batch	req/s	kv/s	MiB/s	p50	p95	p99	校验
4KB	4	2482.98	9931.94	38.80	0.260	0.718	0.781	0 miss / 0 failure

该结果确认 Python binding、master、metadata server 与 Store client 链路全部可用。

真实 read_perf: 吞吐随对象大小变化

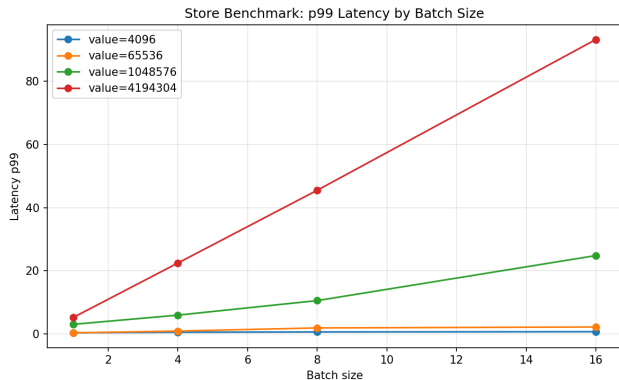


关键观察

- ▶ 小对象受请求开销影响明显。
- ▶ 对象增大后带宽快速上升。
- ▶ 1MB、batch=4 达到峰值 **1399.08 MiB/s**。
- ▶ 4MB 仍有较高吞吐，但尾延迟上升。

value	batch	MiB/s
4KB	16	139.86
64KB	16	791.38
1MB	4	1399.08
4MB	4	1227.74

真实 read_perf: P99 尾延迟权衡



代表性结果

value	batch	p50	p99
4KB	1	0.106	0.380
4KB	16	0.352	0.703
1MB	4	2.574	5.956
4MB	4	12.251	22.456
4MB	16	52.517	93.267

- ▶ batch 与 value size 提升会增加单次传输负载。
- ▶ 4MB + batch=16 的 P99 显著上升。

完整矩阵：16 组真实 Store read_perf

value	batch	MiB/s	p50	p99	状态	value	batch	MiB/s	p50	p99	状态
4KB	1	28.78	0.106	0.380	OK	64KB	1	385.55	0.138	0.354	OK
4KB	4	68.63	0.172	0.539	OK	64KB	4	694.43	0.304	0.917	OK
4KB	8	104.36	0.214	0.626	OK	64KB	8	601.34	0.641	1.901	OK
4KB	16	139.86	0.352	0.703	OK	64KB	16	791.38	0.917	2.199	OK
1MB	1	901.03	1.019	3.046	OK	4MB	1	1084.48	3.623	5.242	OK
1MB	4	1399.08	2.574	5.956	OK	4MB	4	1227.74	12.251	22.456	OK
1MB	8	1342.24	5.224	10.559	OK	4MB	8	1087.55	26.942	45.513	OK
1MB	16	1097.65	11.010	24.824	OK	4MB	16	1015.36	52.517	93.267	OK

return_code 全部为 0

misses 全部为 0

verify_failures 全部为 0

Prefix Reuse 与 Scheduler: 策略评估模块

Prefix Reuse Evaluation

多请求共享 prefix



No Cache: 重复 prefill



Prefix Cache: Store get + decode



输出 TTFT / reduction

Retrieval Scheduler Evaluation

- ▶ metadata: reuse、importance、fetch latency、size、replica。
- ▶ 策略: Random、Nearest、Reuse-aware、CachePilot。
- ▶ 综合得分:

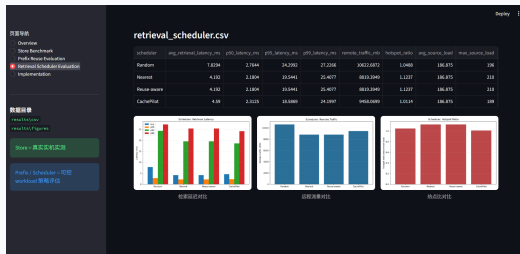
$$score = \alpha \cdot importance + \beta \cdot reuse - \gamma \cdot fetch\ latency$$

Prefix 与 Scheduler 当前定位为基于可控 workload 的离线策略评估, 不宣称为完整分布式 serving 端到端实测。

Dashboard: 面向演示视频与文档截图



Store Benchmark 页面：真实带宽与 P99 趋势



Scheduler 页面：延迟、远程流量、热点比

Dashboard 清楚区分“真实 Store 实机实测”和“Prefix / Scheduler 可控 workload 策略评估”，适合比赛录屏和文档截图。

工程问题与处理记录

问题	表现	处理方式
GLIBCXX_3.4.30 缺失	Python import Mooncake binding 报错	使用系统新版 libstdc++.so.6 进行 LD_PRELOAD
Python binding 与仓库脚本接口差异 4MB 高 batch 初次失败	get_alloc_func_addr 或 nof_replica_num 报错 默认 segment / buffer 偏小	本地实测对官方 benchmark 做兼容补丁 global segment 调至 512MB, local buffer 调至 256MB

这些问题说明：补充 benchmark 文档、依赖说明与自动化脚本，有助于社区复现 Mooncake Store 实验。

当前结论与后续扩展

当前结论

- ▶ Store Benchmark 已完成单机 TCP 实机实测。
- ▶ verify_write 成功, misses=0, verify failures=0。
- ▶ read_perf 16 组全部 return_code=0。
- ▶ 1MB、batch=4 取得 1399.08 MiB/s 峰值带宽。

后续扩展

- ▶ 扩展 RDMA / 多机 / zcopy / write_perf / mixed_rw。
- ▶ 接入 SGLang HiCache 或 vLLM 场景。
- ▶ 用真实 Store latency 标定 Prefix Reuse 与 Scheduler。
- ▶ 以 benchmarks/cachepilot/ 形式准备开源 PR。